Python Engineering - Class 09 Reference Guide

Modules and Imports

Reference guide to the syntax and tools used in this class. It explains what each element does, how it is written and when it is useful.

Quick map

Concept	Syntax	Idea
module	.py file	A Python source file that can expose functions/classes/constants.
import	import module	Load a module and access names through module.name.
from import	from module import f	Import selected names directly.
__name__	special module variable	Equals '__main__' when the file is executed directly; otherwise usually the module nam
main()	def main():	Conventional function that orchestrates application execution.
main guard	if __name__=='__main__':	Execute main only when this file is launched directly.
dependency	import numpy as np	Each module should import what it directly uses.
separation of concerns	physics / processing / orchestration	Each module has one clear responsibility.

1. module

Syntax: `.py file`

A Python source file that can expose functions/classes/constants.

```
# vertical_motion.py

def euler_step(...):
    ...
```

Control question: before running the example, predict the result type and shape.

2. import

Syntax: `import module`

Load a module and access names through module.name.

```
import numpy as np
```

Control question: before running the example, predict the result type and shape.

3. from import

Syntax: `from module import f`

Import selected names directly.

```
from vertical_motion import euler_step
```

Control question: before running the example, predict the result type and shape.

4. __name__

Syntax: `special module variable`

Equals '__main__' when the file is executed directly; otherwise usually the module name.

```
print(__name__)
```

Control question: before running the example, predict the result type and shape.

5. main()

Syntax: `def main():`

Conventional function that orchestrates application execution.

```
def main():
    ...
```

Control question: before running the example, predict the result type and shape.

6. main guard

Syntax: `if __name__=='__main__':`

Execute main only when this file is launched directly.

```
if __name__ == '__main__':
    main()
```

Control question: before running the example, predict the result type and shape.

7. dependency

Syntax: `import numpy as np`

Each module should import what it directly uses.

```
# each module imports what it directly uses
```

Control question: before running the example, predict the result type and shape.

8. separation of concerns

Syntax: `physics / processing / orchestration`

Each module has one clear responsibility.

```
vertical_motion.py -> physics
data_processing.py -> analysis
main.py -> orchestration
```

Control question: before running the example, predict the result type and shape.

Common mistakes

- Confusing a function name with a function call using parentheses.
- Not checking types, indices or shapes before an operation.
- Using a tool because it works without being able to explain what it receives and returns.

Mental cheat sheet

```
module: .py file
import: import module
from import: from module import f
__name__: special module variable
main(): def main():
main guard: if __name__=='__main__':
dependency: import numpy as np
separation of concerns: physics / processing / orchestration
```